

Phase 8.0-8.6 Mock TEE Internal Design

Component Architecture and Data Flow

AegisProof TEE Adapter Layer

Research / PoC Architecture Document

Version 1.0 | August 2026

IMPORTANT

Mock TEE implementation

≠

Production TEE attestation

This document describes research and PoC design only.

1. Overview

This document consolidates the internal design of Phase 8.0 through 8.6 Mock TEE components within the AegisProof TEE Adapter Layer. The objective is to provide an architecture reference that connects cleanly to subsequent phases: Phase 8.7 (Real Acquisition), Phase 8.8 (Verification Stub and Claims Mapping), and Phase 8.9 (Remote Attestation Design and Offline Verification PoC).

Mock TEE is not a substitute for hardware attestation. It is a boundary-validation abstraction used to verify interface correctness, evidence lifecycle, policy evaluation, and deterministic testing without Intel TDX or AMD SEV-SNP hardware.

1.1 Objectives

- Define Phase 8.0-8.6 goals and deliverables
- Explain why Mock TEE was introduced before Real hardware integration
- Document hardware-independent development and evaluation flow
- Establish the Mock to Real TEE migration boundary

1.2 Scope Statement

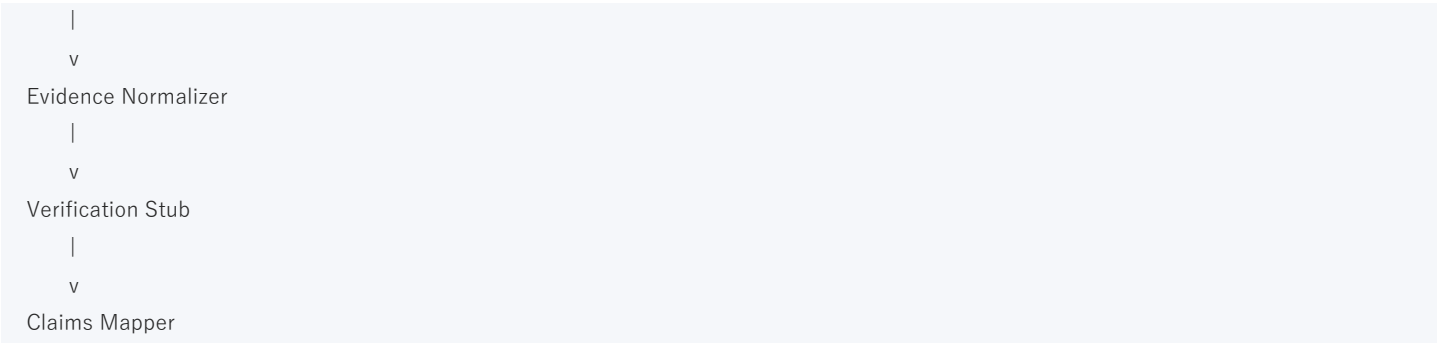
Mock TEE implementation
≠
Production TEE attestation

2. Phase Timeline

Phase	Content	Deliverables
8.0	Mock TEE foundation design	Base interface definitions, Phase 8 architecture scope
8.1	Mock Evidence generation design	Evidence / TeeProvider schema, adapter-layer design
8.2	Provider abstraction	Evaluation framework, mock hardware boundary
8.3	Mock Provider implementation	TdxProviderMock, SevProviderMock
8.4	Evidence normalization and evaluation	EvidenceNormalizer, verification policy tests, Stage A
8.5	Real TEE migration design	Attestation flow design, real evaluation plan
8.6	Parser / Provider / Evaluation foundation	Parsers, ProviderFactory, RealEvidenceNormalizer, Stage B

3. System Architecture





Note: Verification Stub and Claims Mapper were introduced in Phase 8.8. Phase 8.0-8.6 establishes the upstream foundation that these layers extend without breaking Mock separation.

Component	Responsibility	Input	Output	Real TEE Delta
Application Layer	Evaluation and integration requests	Test scenario	Pass/Fail decision	Same entry point
TEE Provider Interface	Abstract generate/verify	reportData Buffer	Evidence object	Unchanged contract
Mock TDX/SEV Provider	Synthetic evidence	Buffer	Evidence + MOCK	Replaced by Real Provider via factory
Mock Evidence Generator	Mock-only factory wrapper	Provider type	Mock TeeProvider	Forces TEE_ENV=mock
Evidence Parser	Structure validation	rawReport bytes	Parsed evidence	Same parsers for Real path
Evidence Normalizer	Canonical mock form	Evidence	NormalizedEvidence	Real uses separate normalizer
Verification Stub	Structure-only verify	rawReport	VerificationResult	Offline/crypto in 8.9B
Claims Mapper	ZK-oriented claims PoC	Normalized evidence	TeeClaims	Protocol non-contact

4. Component Detail

4.1 TEE Provider



The TeeProvider interface (generateEvidence, verifyEvidence, simulateError) absorbs vendor differences between Intel TDX and AMD SEV-SNP. ProviderFactory (Phase 8.6) selects Mock or Real implementations using TEE_ENV. Default remains mock to preserve Stage A.

4.2 Mock Evidence Generator

Generated mock data categories:

- Quote mock (TDX path)
- Report mock (SEV path)
- Measurement (embedded reportData)
- Claims (via policy evaluation)
- Metadata (providerType, isMock, timestamp)

Purpose: parser tests, verification flow tests, integration tests, and Stage A regression.

Mock evidence embeds mandatory flags: MOCK_DATA_ONLY, NOT_REAL_ATTESTATION, DO_NOT_USE_IN_PRODUCTION.

4.3 Parser Layer

In Scope	Out of Scope
Binary/structure parsing	Cryptographic verification
Format validation (version, size)	Trust decision
Field boundary checks	PCCS/KDS collateral fetch

4.4 Evidence Normalizer



Normalized mock evidence fields:

Field	Mock Meaning
identity	provider type (TDX / SEV-SNP)
measurement	Logical placeholder in reportData
claims	Policy-level outcome, not ZK input
timestamp	Generation/evaluation time
metadata	isMock=true, tcbStatus=UpToDate (simulated)

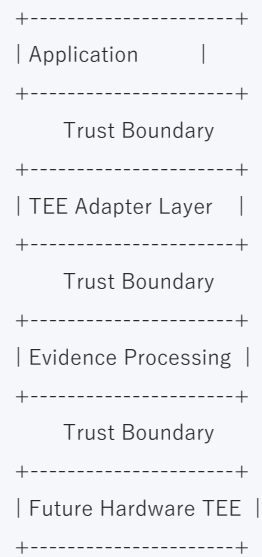
RealEvidenceNormalizer (Phase 8.6) is a separate class with pocScope=structure-only. Mock normalizer rejects non-mock data; Real normalizer rejects mock flags.

5. Data Flow Diagram



Step	Input	Processing	Output	Trust Level
Evidence Generation	reportData	Mock provider adds MOCK flags	Evidence object	Untrusted synthetic
Parser	rawReport bytes	Version/size validation	Validated structure	Structural only
Normalization	Evidence	Mock-only canonical mapping	NormalizedEvidence	Mock domain
Verification Stub	rawReport	Structure-only stub check	VerificationResult	PoC, not production
Application Decision	Policy + results	ZK+TEE composite rules	PASS/FAIL	Evaluation boundary

6. Trust Boundary



Boundary	Trusted	Not Trusted	Production Migration Point
Application	Evaluation scripts, policy config	Provider internals	Unchanged
TEE Adapter	Interface contracts, factory logic	Hardware authenticity	Real Provider swap
Evidence Processing	Normalizer separation rules	Raw bytes origin (mock)	Crypto verification gate
Hardware TEE	Future CPU RoT (not in mock)	Host OS, hypervisor	Phase 8.7+ acquisition

7. Mock vs Real Migration

Layer	Mock (8.0-8.6)	Real (8.7+)
Evidence source	Synthetic (MOCK flags)	Hardware-generated Quote/Report
Parser	Same interface / shared parsers	Same parsers (structure validation)
Provider	Mock adapter	TDX/SEV adapter + acquisition
Verification	Stub / flag check	DCAP/VCEK (offline PoC in 8.9B)
Transport	None	Future Remote Attestation (8.9C)
Normalizer	EvidenceNormalizer (mock only)	RealEvidenceNormalizer (structure-only)

8. Security Considerations

8.1 Prohibited

- Treating mock evidence as production trust
- Using mock key material as cryptographic roots of trust
- Connecting mock path to production verifiers or live PCCS/KDS
- Modifying mock normalizer to accept real hardware evidence

8.2 Maintained Invariants

- Interface separation (TeeProvider contract)
- Provider abstraction (ProviderFactory)
- Normalizer boundary (Mock vs Real classes)
- Protocol isolation (no protocol/circuit changes)

9. Future Phase Connection



Phase 8.0-8.6 remains the regression anchor. All subsequent phases must preserve Stage A (Mock Evaluation) PASS and must not collapse Mock/Real separation.